

Approches classiques de l'IA

Chapitre 5 : Stratégies d'exploration dans les jeux

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 5. Stratégies d'exploration dans les jeux

- 1 Jeux
- 2 Décisions optimales dans les Jeux
- 3 Elagage α - β
- 4 Limites des ressources et évaluation approximative
- 5 Jeux de hasard
- 6 Jeux à information imparfaite
- 7 Résumé du chapitre

Jeux

Un monde multi-agents

Jusqu'à présent, nos agents intelligents évoluaient seuls dans leur environnement.

Que se passe-t-il quand un agent doit considérer **les actions d'autres agents** qui affectent leur bien-être ?

Dans ce chapitre, on se place dans des environnements compétitifs, dans lequel les **objectifs de plusieurs agents sont en conflit**.

On étudie des **problèmes d'exploration conflictuelle** (adversarial search problems), aussi connus sous le nom de **jeux**.

Jeux

En **théorie (mathématique) des jeux** : tout environnement multi-agents est vu comme un **jeu** pour peu que le choix d'un agent (protagoniste) a un impact sur les autres agents.

Nous commencerons par nous intéresser à des jeux d'un type bien précis :

- Un théoricien de jeux dirait :
- Nous dirions :

Jeux

En **théorie (mathématique) des jeux** : tout environnement multi-agents est vu comme un **jeu** pour peu que le choix d'un agent (protagoniste) a un impact sur les autres agents.

Nous commencerons par nous intéresser à des jeux d'un type bien précis :

- Un théoricien de jeux dirait : déterministe (sans hasard ou chance),
- Nous dirions : dans un environnement déterministe

Jeux

En **théorie (mathématique) des jeux** : tout environnement multi-agents est vu comme un **jeu** pour peu que le choix d'un agent (protagoniste) a un impact sur les autres agents.

Nous commencerons par nous intéresser à des jeux d'un type bien précis :

- Un théoricien de jeux dirait : déterministe (sans hasard ou chance), à information parfaite
- Nous dirions : dans un environnement déterministe et complètement observable

Jeux

En **théorie (mathématique) des jeux** : tout environnement multi-agents est vu comme un **jeu** pour peu que le choix d'un agent (protagoniste) a un impact sur les autres agents.

Nous commencerons par nous intéresser à des jeux d'un type bien précis :

- Un théoricien de jeux dirait : déterministe (sans hasard ou chance), à information parfaite, à 2 joueurs,
- Nous dirions : dans un environnement déterministe et complètement observable où deux agents agissent

Jeux

En **théorie (mathématique) des jeux** : tout environnement multi-agents est vu comme un **jeu** pour peu que le choix d'un agent (protagoniste) a un impact sur les autres agents.

Nous commencerons par nous intéresser à des jeux d'un type bien précis :

- Un théoricien de jeux dirait : déterministe (sans hasard ou chance), à information parfaite, à 2 joueurs, tour-à-tour
- Nous dirions : dans un environnement déterministe et complètement observable où deux agents agissent en alternance

Jeux

En **théorie (mathématique) des jeux** : tout environnement multi-agents est vu comme un **jeu** pour peu que le choix d'un agent (protagoniste) a un impact sur les autres agents.

Nous commencerons par nous intéresser à des jeux d'un type bien précis :

- Un théoricien de jeux dirait : déterministe (sans hasard ou chance), à information parfaite, à 2 joueurs, tour-à-tour et à somme nulle.
- Nous dirions : dans un environnement déterministe et complètement observable où deux agents agissent en alternance et où la somme des valeurs de la fonction d'utilité (en fin de partie) des agents est constante, quelle que soit le déroulement de la partie.

Jeux

Remarques :

- “somme nulle” : (= somme constante) par exemple si un joueur gagne $[+1]$ (la partie d'échecs), alors l'autre perd nécessairement $[0]$. En cas d'égalité, les 2 joueurs obtiennent $[\frac{1}{2}]$. Dans tous les cas, la somme vaut toujours $[1]$.
- Nous verrons comment lever certaines de ces hypothèses (2 joueurs ; déterministe ; information parfaite).
- Dans ce chapitre (et ce cours), on ne parle donc pas de l'IA dans des jeux vidéos (comme le comportement de “PNJ” – personnage non joueur)

Types de jeux

Quelques exemples de jeux.

Information	Déterministe	Avec chance
parfaite	Echecs, Go, Othello	Backgammon, Monopoly
imparfaite	Combat Naval	Bridge, Poker



Types de jeux

Quelques exemples de jeux.

Information	Déterministe	Avec chance
parfaite	Echecs, Go, Othello	Backgammon, Monopoly
imparfaite	Combat Naval	Bridge, Poker



Types de jeux

Quelques exemples de jeux.

Information	Déterministe	Avec chance
parfaite	Echecs, Go, Othello	Backgammon, Monopoly
imparfaite	Combat Naval	Bridge, Poker



Types de jeux

Quelques exemples de jeux.

Information	Déterministe	Avec chance
parfaite	Echecs, Go, Othello	Backgammon , Monopoly
imparfaite	Combat Naval	Bridge, Poker



Types de jeux

Quelques exemples de jeux.

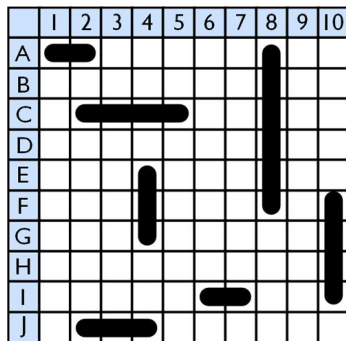
Information	Déterministe	Avec chance
parfaite	Echecs, Go, Othello	Backgammon, Monopoly
imparfaite	Combat Naval	Bridge, Poker



Types de jeux

Quelques exemples de jeux.

Information	Déterministe	Avec chance
parfaite	Echecs, Go, Othello	Backgammon, Monopoly
imparfaite	Combat Naval	Bridge, Poker



Types de jeux

Quelques exemples de jeux.

Information	Déterministe	Avec chance
parfaite	Echecs, Go, Othello	Backgammon, Monopoly
imparfaite	Combat Naval	Bridge, Poker



Types de jeux

Quelques exemples de jeux.

Information	Déterministe	Avec chance
parfaite	Echecs, Go, Othello	Backgammon, Monopoly
imparfaite	Combat Naval	Bridge, Poker



Jeux et IA

Pour les chercheurs en IA, les jeux sont pratiques de part leur nature **abstraite** :

- état d'un jeu facile à représenter ;
- nombre d'actions limitées ;
- règles précises, etc.

Jeux et IA

D'autre part, les jeux sont **intéressants** car

- **difficiles à résoudre** : par ex., l'arbre de recherche du jeu d'échecs $\simeq 10^{154}$ noeuds ;
- demandent une **décision rapide** : approximation car optimalité impossible en pratique ;
- **sanctionnent sévèrement l'inefficacité** : une exploration A^* à moitié efficace prendra deux fois plus de temps à être résolue ; un programme d'échecs qui n'utilise qu'à moitié le temps qu'il lui est imparti a de fortes chances d'échouer.

⇒ comment utiliser au mieux le temps (les ressources) ?

Formulation du problème

Un jeu est un **problème d'exploration** particulier. Il est défini par :

- S_0 : l'**état initial** (comment le jeu est-il mis en place)
- $\text{PLAYER}(s)$: définit **quel joueur** va jouer le prochain tour à partir d'un état s
- $\text{ACTION}(s)$: retourne l'ensemble des **actions autorisées** depuis l'état s
- $\text{RESULT}(s, a)$: **modèle de transition** qui définit le résultat d'une action a depuis l'état s
- $\text{TERMINAL-TEST}(s)$: retourne vrai si et seulement si le jeu est terminé en s (états terminaux)
- $\text{UTILITY}(s, p)$: une **fonction d'utilité** définissant une valeur numérique finale (le **profit**, payoff) pour un jeu qui se termine dans un état s pour un joueur p

Arbre d'exploration

L'arbre d'exploration d'un jeu est défini par

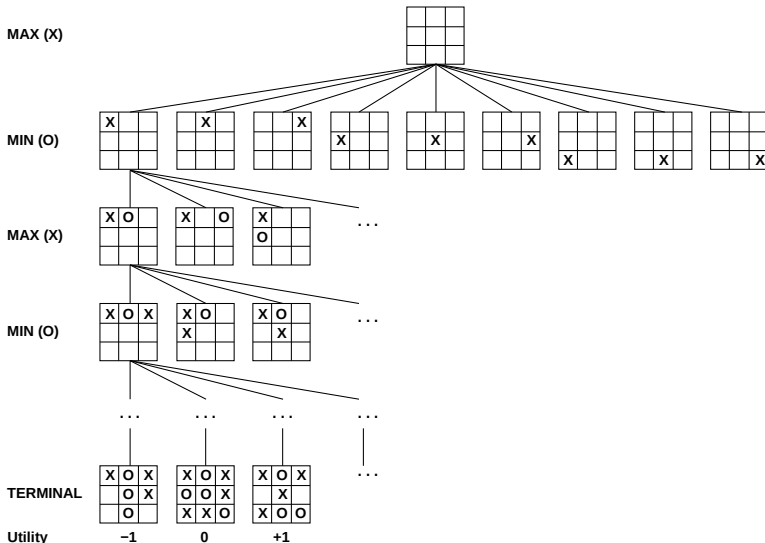
- l'état initial S_0
- la fonction ACTIONS
- la fonction RESULT

Les noeuds d'un arbre d'exploration correspondent aux états possibles du jeu.

Dans les jeux à 2 joueurs, on appelle MAX le premier joueur et MIN le second joueur.

Arbre d'exploration

Exemple : OXO (2 joueurs, déterministe, tour à tour, information parfaite, utilité selon MAX)



Qui commence ?

Pourquoi peut-on toujours considérer que l'agent IA joue toujours MAX et donc joue en **premier** (racine de l'arbre d'exploration) ?

Qui commence ?

Pourquoi peut-on toujours considérer que l'agent IA joue toujours MAX et donc joue en **premier** (racine de l'arbre d'exploration) ?

Car l'état initial S_0 est l'état "initial" par rapport au moment où **l'agent doit prendre sa prochaine décision** (peu importe si le jeu à déjà commencé ou non, une fois que c'est au tour de l'agent de jouer, l'état courant du jeu est l'état initial de son exploration).

Décisions optimales dans les Jeux

Minimax

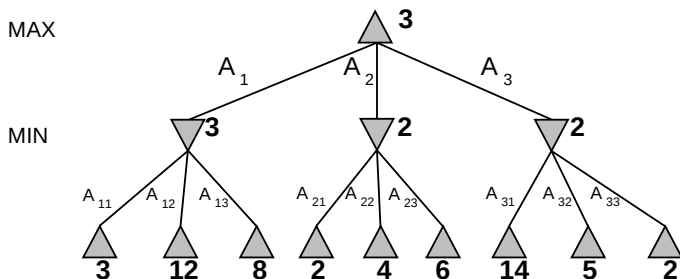
Comment jouer de façon optimale dans un jeu déterministe, avec information parfaite ?

Idée : choisir l'action qui nous place en l'état qui possède la plus haute valeur **minimax**.

minimax = meilleur profit possible contre un adversaire qui joue de manière optimale

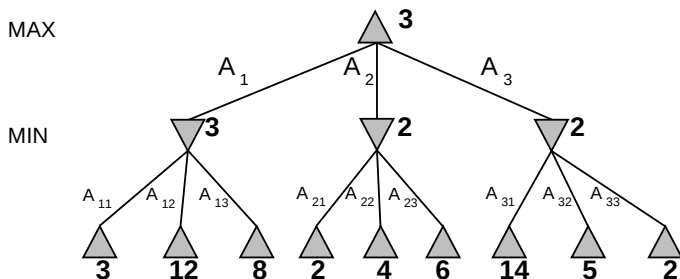
Minimax

Exemple : jeu à un seul tour ou deux “plis” (chaque joueur joue une seule fois). Quel est le meilleur coup pour MAX ?



Minimax

Exemple : jeu à un seul tour ou deux “plis” (chaque joueur joue une seule fois). Quel est le meilleur coup pour MAX ?



Le meilleur coup à jouer pour MAX est A_1 car il lui assure le meilleur profit si MIN joue de manière optimale.

Minimax

Valeur **minimax** d'un noeud = utilité pour MAX d'être dans l'état correspondant, si les 2 joueurs jouent de manière optimale.

MINIMAX(s) =

$$\begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{si } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{ACTIONS}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{si } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{ACTIONS}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{si } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

⇒ d'où la terminologie MAX et MIN pour les joueurs.

Algorithme Minimax

```
function MINIMAX-DECISION(state) returns an action
  return  $a \in \text{ACTIONS}(\textit{state})$  maximizing MIN-VALUE(RESULT(state, a))
```

```
function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state, MAX)
   $v \leftarrow -\infty$ 
  for a in ACTIONS(state) do
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(\textit{state}, a)))$ 
  return v
```

```
function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state, MAX)
   $v \leftarrow \infty$ 
  for a in ACTIONS(state) do
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(\textit{state}, a)))$ 
  return v
```

Propriétés de l'algorithme Minimax

L'algorithme Minimax est **optimal** contre un adversaire optimal.

Et si l'adversaire ne joue pas de manière optimale ?

Propriétés de l'algorithme Minimax

L'algorithme Minimax est **optimal** contre un adversaire optimal.

Et si l'adversaire ne joue pas de manière optimale ?

- Minimax maximise le profit de MAX **dans le pire des cas** (quand MIN joue de manière optimale).
- Si MIN ne joue pas de manière optimale : MAX peut faire encore mieux.
- D'autres stratégies peuvent faire mieux que la stratégie Minimax contre des adversaires sous-optimaux ;
- **mais** ces stratégies font nécessairement moins bien contre des adversaires optimaux.

Propriétés de l'algorithme Minimax

Cet algorithme = DFS (version récursive) complet de l'arbre d'exploration.

Si b est le nombre (maximum) d'actions possibles à chaque état et m est la profondeur maximum de l'arbre, on a (cf. Chap. 3) :

Temps ?? $O(b^m)$

Espace ?? $O(bm)$

Propriétés de l'algorithme Minimax

Cet algorithme = DFS (version récursive) complet de l'arbre d'exploration.

Si b est le nombre (maximum) d'actions possibles à chaque état et m est la profondeur maximum de l'arbre, on a (cf. Chap. 3) :

Temps ?? $O(b^m)$

Espace ?? $O(bm)$

Jouer de manière optimale est donc irréalisable en pratique.

Exemple : Aux échecs, $b \approx 35$, $m \approx 100$ pour des parties “raisonnables”

$$35^{100} \simeq 2.5 \times 10^{154}$$

Jeux à plus de 2 joueurs

Beaucoup de jeux se jouent à plus de 2 joueurs. Comment adapter l'idée de l'algorithme Minimax à plus de 2 joueurs ?

Jeux à plus de 2 joueurs

Beaucoup de jeux se jouent à plus de 2 joueurs. Comment adapter l'idée de l'algorithme Minimax à plus de 2 joueurs ?

Facile d'un point de vue technique :

- **Utilité** : remplacer chaque valeur simple associée à un noeud par un **vecteur de valeurs**.
 - ▶ **Exemple** : 3 joueurs $A, B, C \Rightarrow$ on associe un vecteur $\langle v_A, v_B, v_C \rangle$ à chaque noeud.
 - ▶ **En pratique** : le plus simple est d'adapter UTILITY pour retourner un vecteur d'utilités
 - ▶ **Pourquoi une seule valeur à 2 joueurs ?**

Jeux à plus de 2 joueurs

Beaucoup de jeux se jouent à plus de 2 joueurs. Comment adapter l'idée de l'algorithme Minimax à plus de 2 joueurs ?

Facile d'un point de vue technique :

- **Utilité** : remplacer chaque valeur simple associée à un noeud par un **vecteur de valeurs**.
 - ▶ **Exemple** : 3 joueurs $A, B, C \Rightarrow$ on associe un vecteur $\langle v_A, v_B, v_C \rangle$ à chaque noeud.
 - ▶ **En pratique** : le plus simple est d'adapter UTILITY pour retourner un vecteur d'utilités
 - ▶ **Pourquoi une seule valeur à 2 joueurs ?** Dans un jeu à somme nulle, la deuxième composante du vecteur est implicite.

Jeux à plus de 2 joueurs

- **Adaptation de l'idée Minimax** : à un noeud donné n , comment sélectionner le vecteur d'utilité parmi les descendants ?
 - ▶ A 2 joueurs, ...
 - ▶ A plus de 2 joueurs, ...

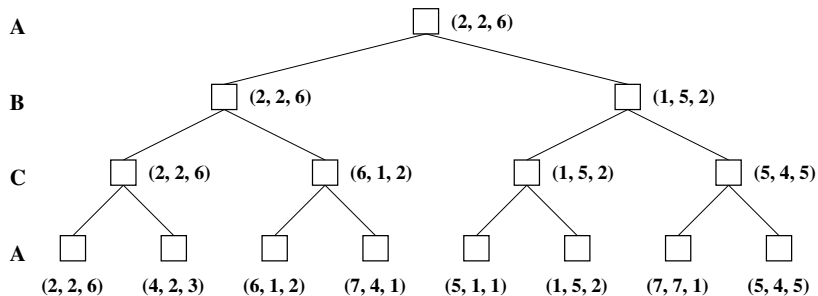
Jeux à plus de 2 joueurs

- **Adaptation de l'idée Minimax** : à un noeud donné n , comment sélectionner le vecteur d'utilité parmi les descendants ?
 - ▶ A 2 joueurs, MAX et MIN choisissent la plus **haute valeur** d'utilité (selon leur propre point de vue : MIN prend l'utilité opposée car jeu à somme nulle).
 - ▶ A plus de 2 joueurs, ...

Jeux à plus de 2 joueurs

- **Adaptation de l'idée Minimax** : à un noeud donné n , comment sélectionner le vecteur d'utilité parmi les descendants ?
 - ▶ A 2 joueurs, MAX et MIN choisissent la plus **haute valeur** d'utilité (selon leur propre point de vue : MIN prend l'utilité opposée car jeu à somme nulle).
 - ▶ A plus de 2 joueurs, pour un noeud associé au joueur n , il choisira le vecteur (parmi ses fils) dont sa propre composante à une valeur la plus haute.

Jeux à plus de 2 joueurs



Jeux à plus de 2 joueurs : alliances et trahisons

D'un point de vue plus conceptuel que technique, les jeux à plus de 2 joueurs posent des questions intéressantes (hors cadre de ce cours), comme les alliances, la coopération temporaire, la trahison (bris d'alliance), etc.

Exemple : les alliances sont-elles une conséquence naturelle de stratégies optimales pour chaque joueur ?

Jeux à plus de 2 joueurs : alliances et trahisons

D'un point de vue plus conceptuel que technique, les jeux à plus de 2 joueurs posent des questions intéressantes (hors cadre de ce cours), comme les alliances, la coopération temporaire, la trahison (bris d'alliance), etc.

Exemple : les alliances sont-elles une conséquence naturelle de stratégies optimales pour chaque joueur ?

Remarque : si un jeu à 2 joueurs n'est pas à somme nulle, alors des stratégies collaboratives peuvent apparaître.

Exemple : un jeu où l'utilité maximum est 1000 et où il existe un état terminal idéal ayant comme utilité le vecteur $\langle 1000, 1000 \rangle$.

Elagage α - β

Faut-il explorer tout l'arbre ?

L'algorithme Minimax est en $O(b^m)$. Mais doit-on nécessairement explorer tous les chemins ?

Faut-il explorer tout l'arbre ?

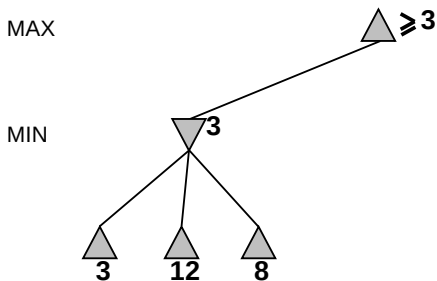
L'algorithme Minimax est en $O(b^m)$. Mais doit-on nécessairement explorer tous les chemins ?

Non : il est possible de calculer la décision minimax correcte sans explorer l'entièreté de l'arbre.

Idée : élaguer des branches de l'arbre du jeu qui ne sont pas nécessaires au calcul du minimax de la racine.

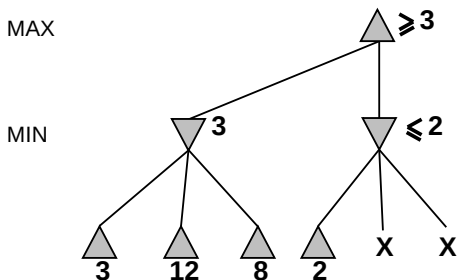
⇒ **Elagage α - β** (α - β pruning)

Exemple d'élagage



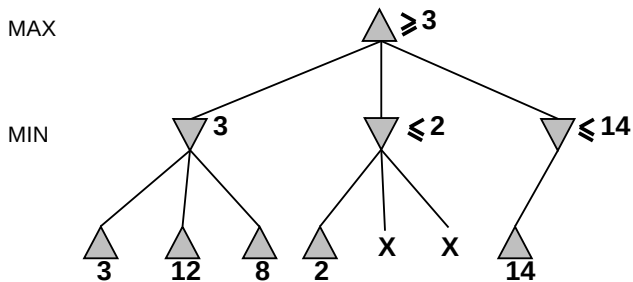
A ce niveau du minimax, l'on sait déjà que $\text{MINIMAX}(\text{root}) \geq 3$.

Exemple d'élagage

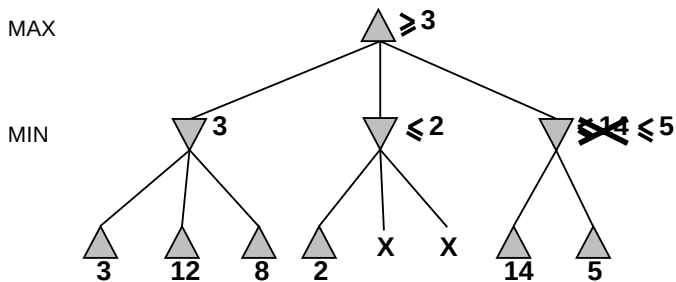


Le deuxième fils de *root* sera ≤ 2 puisque son premier descendant vaut 2 (et que c'est un noeud MIN). Il n'est pas nécessaire de continuer l'exploration de ses descendants : un jeu optimal ne les atteindra jamais car MAX ne choisira pas l'action qui y mène : on sait déjà qu'il a une meilleure alternative.

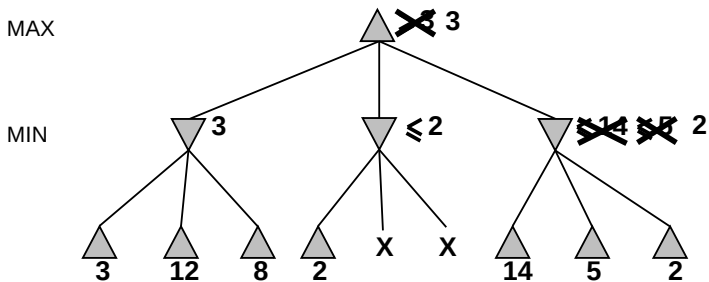
Exemple d'élagage



Exemple d'élagage



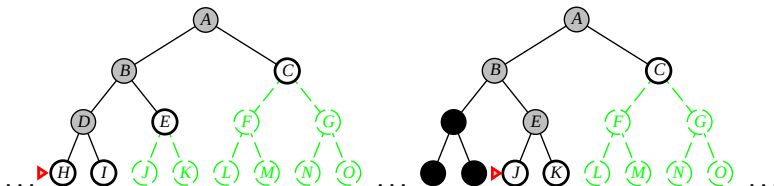
Exemple d'élagage



Principe de l'élagage α - β

Principe : appliquer un MINIMAX et maintenir deux valeurs (α et β) permettant de savoir si on peut élaguer une branche.

Rappel (cf. Chap. 3) : Minimax est un DFS $\Rightarrow$ les noeuds qui n'ont pas encore été complètement explorés se trouvent sur un seul chemin (noeuds en gris ci-dessous). Le noeud en cours de considération est au bout de ce chemin. Dans l'interprétation de α et β , on considère uniquement ce chemin.



Principe de l'élagage α - β

En fonction de ce qui a déjà été exploré,

- α = valeur de la meilleure alternative pour MAX le long du chemin jusqu'à l'état considéré (= plus haute valeur d'un fils MIN déjà exploré)

L'exploration α - β met à jour les valeur de α et β et élague les branches non explorées du noeud courant n si :

- n est MIN et un de ses fils (MAX) a une valeur $\leq \alpha$ (coupe α) ;

Principe de l'élagage α - β

En fonction de ce qui a déjà été exploré,

- α = valeur de la meilleure alternative pour MAX le long du chemin jusqu'à l'état considéré (= plus haute valeur d'un fils MIN déjà exploré)
- β = valeur de la meilleure alternative pour MIN le long du chemin jusqu'à l'état considéré (= plus petite valeur d'un fils MAX déjà exploré)

L'exploration α - β met à jour les valeur de α et β et élague les branches non explorées du noeud courant n si :

- n est MIN et un de ses fils (MAX) a une valeur $\leq \alpha$ (coupe α) ;
- n est MAX et un de ses fils (MIN) a une valeur $\geq \beta$ (coupe β).

Algorithme α - β

(Equivalent à Minimax + mise à jour valeurs α et β + élagages)

function ALPHA-BETA-SEARCH(*state*) **returns** an action

$v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$

return the *action* in ACTIONS(*state*) with value v

function MAX-VALUE(*state*, α , β) **returns** a utility value

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*, MAX)

$v \leftarrow -\infty$

for each a **in** ACTIONS(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$

if $v \geq \beta$ **then return** v

$\alpha \leftarrow \text{MAX}(\alpha, v)$

return v

...

Algorithme α - β

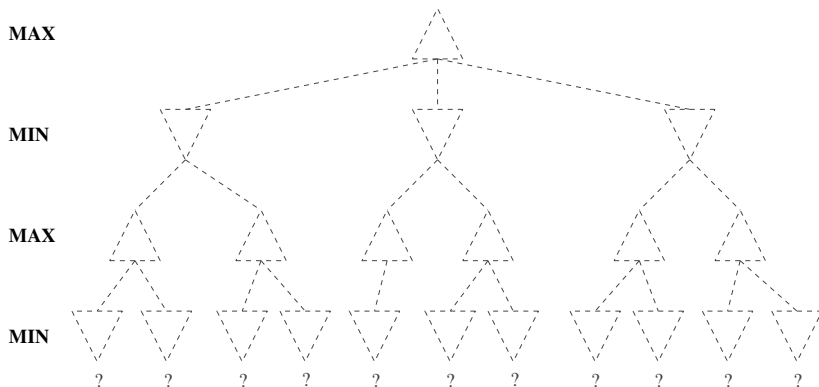
...

```

function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state, MAX)
   $v \leftarrow +\infty$ 
  for each a in ACTIONS(state) do
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$ 
    if  $v \leq \alpha$  then return  $v$ 
     $\beta \leftarrow \text{MIN}(\beta, v)$ 
  return  $v$ 

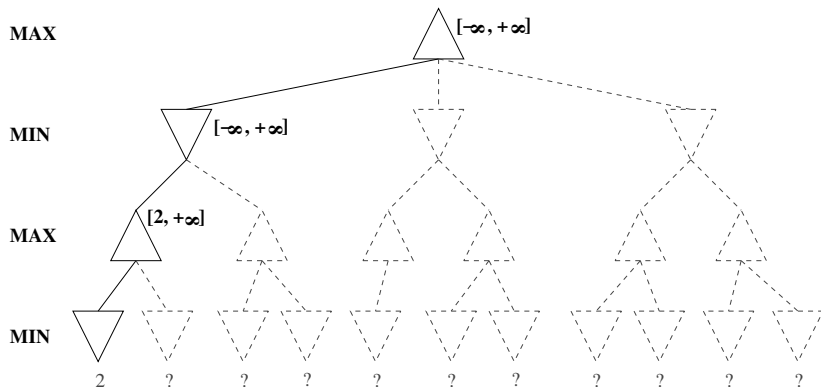
```

Algorithme α - β en action



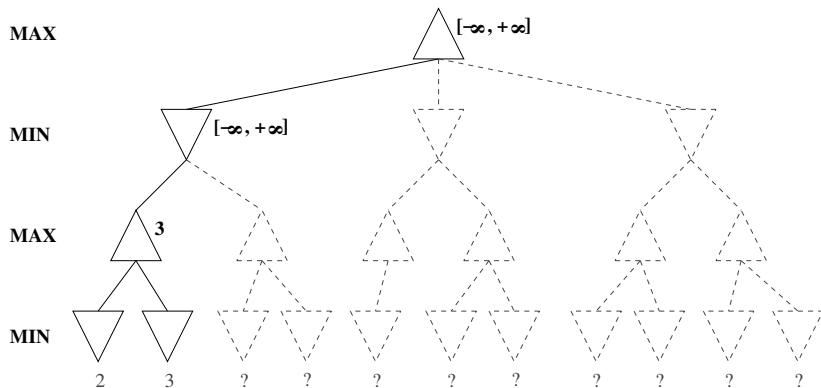
Application de la recherche α - β sur un jeu à 2 joueurs en 3 plis.

Algorithme α - β en action



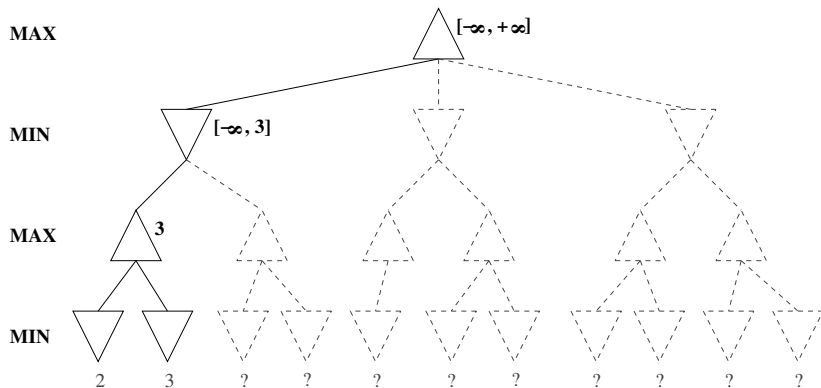
Première descente en profondeur jusqu'à un noeud terminal. Les valeurs $[\alpha, \beta]$ connues jusqu'à présent peuvent être vues comme des **fenêtres** sur la valeur Minimax exacte d'un noeud (bornes inférieure et supérieure).

Algorithme α - β en action



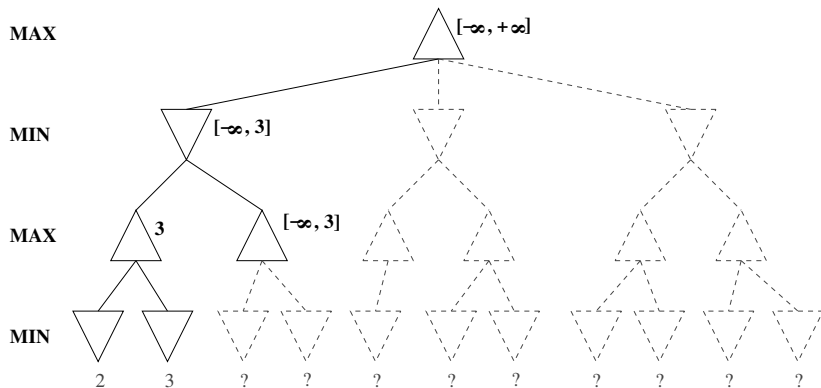
La valeur du noeud MAX en bas à gauche est entièrement calculée et vaut 3.

Algorithme α - β en action



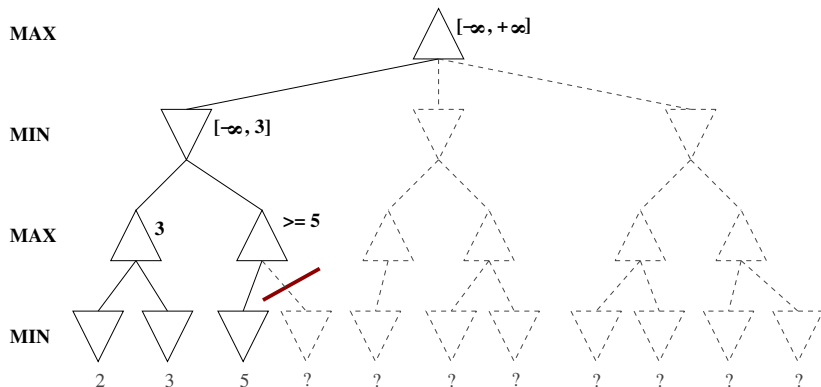
Cette information remonte dans le noeud MIN parent qui avait appelé MAX-VALUE (dans un appel MIN-VALUE) ; et qui met à jour sa valeur β .

Algorithme α - β en action



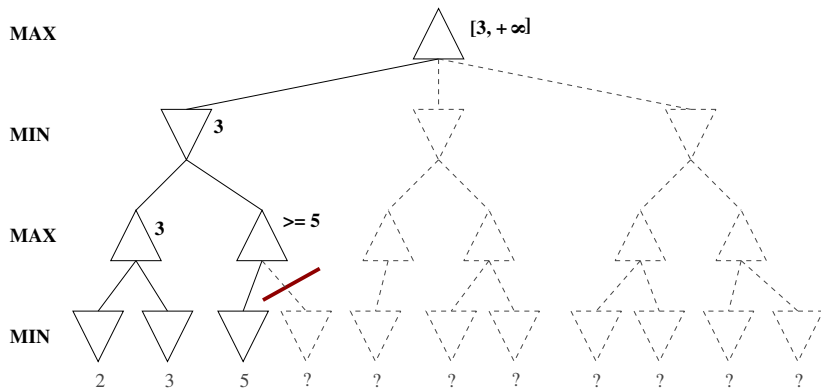
L'exploration continue en suivant l'ordre d'un DFS (défini par les appels récursifs alternés en MAX-VALUE et MIN-VALUE). Les valeurs $[\alpha, \beta]$ sont passées comme paramètres de ces appels récursifs alternés.

Algorithme α - β en action



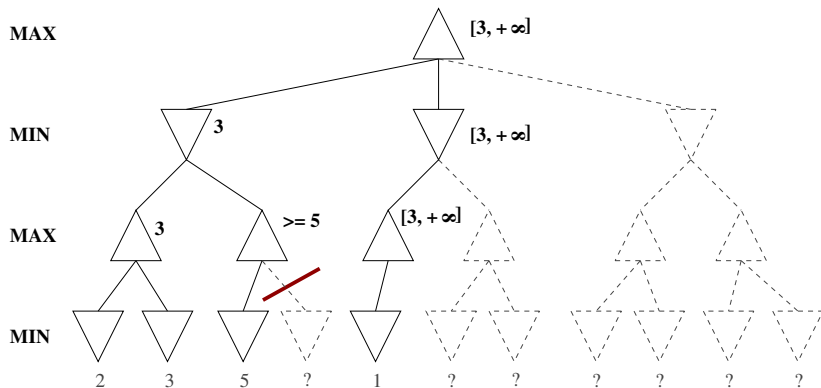
MAX-VALUE sur le noeud courant va affecter 5 à la variable v . Le test “**if** $v \geq \beta$ ” sera vrai et les autres descendants seront élagués. L’appel à MAX-VALUE retourne directement 5, ce qui signifie : rien ne sert d’aller dans ce noeud avec une valeur ≥ 5 , on peut faire mieux ailleurs ($\beta = 3$) $\Rightarrow$ **coupe** β

Algorithme α - β en action



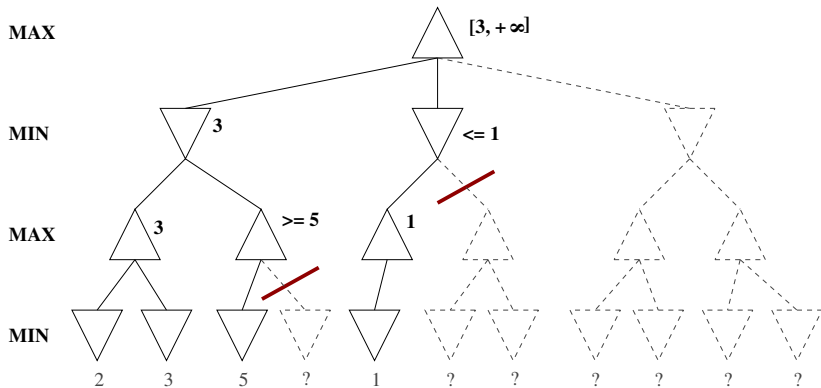
La valeur Minimax exacte du fils gauche de la racine est connue et vaut 3.
 Cette information est passée à la racine qui met sa valeur α à jour.

Algorithme α - β en action



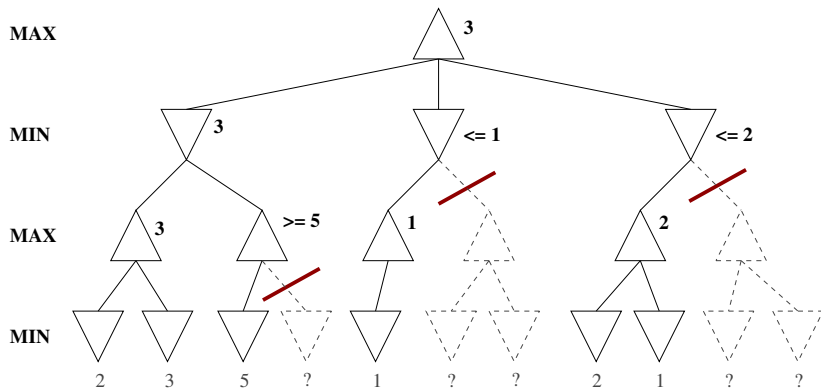
Descente en profondeur via le deuxième fils de la racine. Un noeud terminal d'utilité 1 est trouvé.

Algorithme α - β en action



L'information remonte vers le deuxième fils de la racine : la valeur Minimax de son fils gauche est connue et vaut 1. Dans MIN-VALUE, cette valeur est affectée à la variable v et le test "**if** $v \leq \alpha$ " est vrai ($\alpha = 3$). Cela signifie : rien ne sert d'explorer les autres descendants, on peut faire mieux ailleurs ($\alpha = 3$) $\Rightarrow$ coupe α .

Algorithme α - β en action



Un peu plus loin lors de l'exploration, une deuxième coupe α est effectuée. L'exploration est terminée : environ la moitié des noeuds terminaux ont été visités.

Propriétés de l'algorithme α - β

- l'élagage **n'affecte pas** le résultat final ;
- l'ordre de sélection des actions a une influence sur l'efficacité de l'élagage ;
- avec un “ordre parfait”, la complexité en temps = $O(b^{m/2})$;
- on “double” la profondeur calculable mais reste impossible en pratique pour beaucoup de jeux ;
- $\Rightarrow$ heuristiques de type “best-move first” pour s'approcher de l'ordre parfait ;
- variantes existent : par exemple, adaptation du DFS récursive (et donc Tree-Search) avec Iterative Deepening Search ou Graph Search (voir Chapitre 3).

Limites des ressources et évaluation approximative

Limites des ressources

But : les approches optimales sont impraticables. Comment prendre une bonne décision en limitant les ressources (principalement le **temps** mis pour prendre une décision) ?

Limites des ressources

But : les approches optimales sont impraticables. Comment prendre une bonne décision en limitant les ressources (principalement le **temps** mis pour prendre une décision) ?

Approche classique :

- Utiliser $\text{CUTOFF-TEST}(s, d)$ à la place de $\text{TERMINAL-TEST}(s)$: typiquement, un test qui **limite la profondeur** d de l'exploration ;
- Utiliser $\text{EVAL}(s, p)$ à la place de $\text{UTILITY}(s, p)$: **fonction d'évaluation** qui estime la qualité d'une position dans le jeu (appelée quand CUTOFF-TEST est vrai)

Limites des ressources

But : les approches optimales sont impraticables. Comment prendre une bonne décision en limitant les ressources (principalement le **temps** mis pour prendre une décision) ?

Approche classique :

- Utiliser CUTOFF-TEST(s, d) à la place de TERMINAL-TEST(s) : typiquement, un test qui **limite la profondeur** d de l'exploration ;
- Utiliser EVAL(s, p) à la place de UTILITY(s, p) : **fonction d'évaluation** qui estime la qualité d'une position dans le jeu (appelée quand CUTOFF-TEST est vrai)

Exemple (échecs) : on dispose de **100** sec et on peut explorer **10^4** noeuds/sec

⇒ **10^6** noeuds par action $\approx$ **$35^{8/2}$**

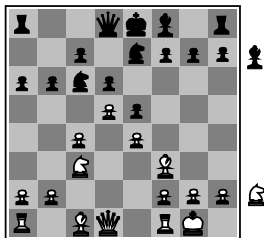
⇒ α - β avec profondeur 8 ⇒ très bon programme

Fonctions d'évaluation

La fonction EVAL est

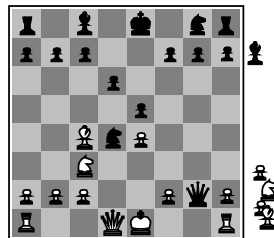
- en général la clef des bons programmes d'IA dans les jeux ;
- très dépendante du jeu ;
- exprime souvent ce que de bons joueurs (humains) estiment comme une bonne position dans le jeu.
- équivalente aux fonctions heuristiques d'une exploration (cf. Chap. 3) :
 - ▶ fonction heuristique : estimation de la **distance** d'un état donné vers un état objectif ;
 - ▶ fonction EVAL : estimation de l'**utilité** espérée à partir d'une position du jeu donnée.

Fonctions d'évaluation : échecs



Black to move

White slightly better



White to move

Black winning

Aux échecs, typiquement une somme pondérée de **caractéristiques** f_i :

$$\text{EVAL}(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s).$$

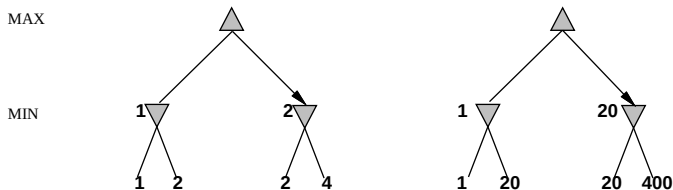
Par ex. $w_1 = 9$ avec $f_1(s) = (\# \text{ de reines blanches}) - (\# \text{ de reines noires})$, etc.

Fonctions d'évaluation : Othello

A Othello, typiquement une somme pondérée des **cases en sa possession** (par ex. les coins sont des cases primordiales car sont définitivement acquises et permettent l'extension de sa couleur : poids important). Le poids d'une case évolue au cours de la partie.

120	-20	20	5	5	20	-20	120
-20	-40	-5	-5	-5	-5	-40	-20
20	-5	15	3	3	15	-5	20
5	-5	3	3	3	3	-5	5
5	-5	3	3	3	3	-5	5
20	-5	15	3	3	15	-5	20
-20	-40	-5	-5	-5	-5	-40	-20
120	-20	20	5	5	20	-20	120

Fonctions d'évaluation : les valeurs exactes n'ont pas d'importance



Le comportement est préservé pour toute transformation **monotone** de EVAL.

Seul l'ordre importe ici : les profits dans les jeux **déterministes** $\simeq$ fonction d'**utilité ordinale**.

Attention : cela sera différent avec le hasard ! (voir plus loin)

Couper la recherche (CUTOFF-TEST)

Pour limiter la recherche

- le moyen le plus simple : CUTOFF-TEST basé sur une profondeur maximum (estimée en fonction des ressources) ;
- moyen plus robuste : iterative deepening search (cf. Chap. 3) stoppé quand ressources maximum atteintes (retourne l'action correspondant à la recherche complète la plus profonde). Bonus : IDS aide également pour le problème de l'ordre des actions (comme un BFS).

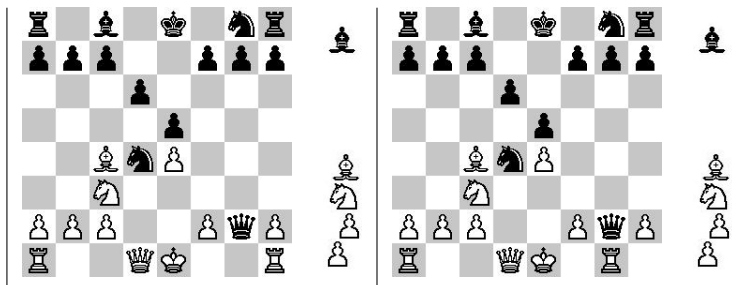
Couper la recherche (CUTOFF-TEST)

Pour limiter la recherche

- le moyen le plus simple : CUTOFF-TEST basé sur une profondeur maximum (estimée en fonction des ressources) ;
- moyen plus robuste : iterative deepening search (cf. Chap. 3) stoppé quand ressources maximum atteintes (retourne l'action correspondant à la recherche complète la plus profonde). Bonus : IDS aide également pour le problème de l'ordre des actions (comme un BFS).

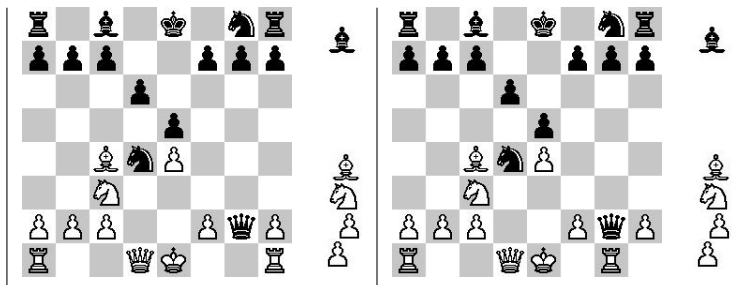
Mais ces approches simples peuvent conduire à des erreurs dues à l'approximation de la fonction EVAL.

Couper la recherche (CUTOFF-TEST) : ex. d'erreur dûe à EVAL



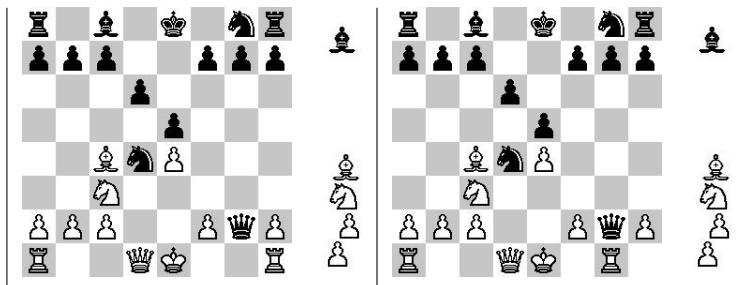
- les deux situations sont identiques sauf pour la position de la tour blanche en bas à droite.
- une fonction d'évaluation uniquement basée sur les pièces en jeu aurait la même valeur (à l'avantage des noirs)
- pourtant, la situation de droite est très désavantageuse pour les noirs : au coup suivant, Blanc prend la reine noire

Couper la recherche (CUTOFF-TEST) : ex. d'erreur dûe à EVAL



- les deux situations sont identiques sauf pour la position de la tour blanche en bas à droite.
- une fonction d'évaluation uniquement basée sur les pièces en jeu aurait la même valeur (à l'avantage des noirs)
- pourtant, la situation de droite est très désavantageuse pour les noirs : au coup suivant, Blanc prend la reine noire

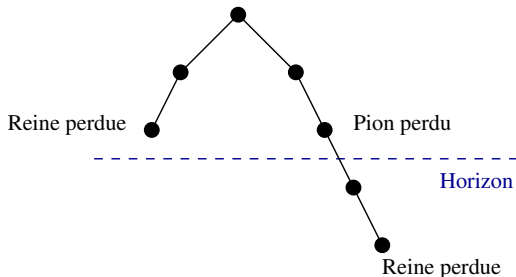
Couper la recherche (CUTOFF-TEST) : ex. d'erreur dûe à EVAL



- les deux situations sont identiques sauf pour la position de la tour blanche en bas à droite.
- une fonction d'évaluation uniquement basée sur les pièces en jeu aurait la même valeur (à l'avantage des noirs)
- pourtant, la situation de droite est très désavantageuse pour les noirs : au coup suivant, Blanc prend la reine noire

Couper la recherche (CUTOFF-TEST) : effet horizon

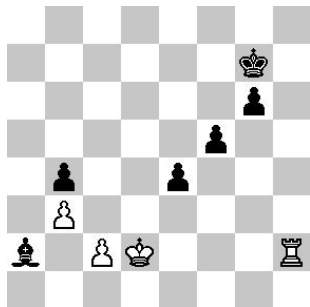
Autre problème : effet horizon dû a la limite sur la profondeur.



Le programme essaie d'éviter une perte sérieuse, mais ne fait que la postposer (au delà de ce qu'il "voit"), souvent au prix de sacrifice(s) inutile(s)

Couper la recherche (CUTOFF-TEST) : effet horizon

Exemple



- en 6 plis, le fou noir coincé peut être pris par la tour blanche
- mais Noir peut repousser la capture du fou au delà de son horizon
- par ex., si $d = 8$ plis, la plupart des mouvements de Noir : capture du fou = rejetés
- Noir va mettre Blanc en échec avec un pion ; qui sera sacrifié (2 plis)
- il va répéter ce sacrifice avec le pion suivant (2 plis)
- les 4 plis restants ne sont pas suffisants pour voir que la situation du fou noir n'a pas changé

Couper la recherche (CUTOFF-TEST)

En pratique, il vaut mieux appliquer un CUTOFF-TEST plus sophistiqué qu'une simple limite sur la profondeur.

Exemples :

- **Quiescent search** (recherche calme) : ne couper que les états qui ne montrent pas de grands changements dans la fonction d'évaluation (minimise les erreurs dues à EVAL)
- **Singular extension** : retenir des mouvements considérés comme bons, et ne pas limiter la recherche à d pour ceux-ci (atténue l'effet horizon)*
- **Forward pruning** : couper les mouvements qui sont considérés comme mauvais*

(* nécessite de l'expertise par rapport au jeu ou de l'apprentissage automatique)

Jeux déterministes : bref état de l'art

- **Echecs** : DEEP BLUE a battu le champion du monde Kasparov en 1997 dans une partie en 6 manches.
 - ▶ α - β sur ordinateur parallèle avec 30 processeurs
 - ▶ + une configuration spéciale de 480 processeurs spécifiquement dédiés aux échecs pour génération et ordonnancement des derniers niveaux de l'arbre
 - ▶ 30 milliards de noeuds évalués par tour dans le temps imparti $\Rightarrow$ profondeur de 14
 - ▶ méthodes (non divulguées) pour atteindre une profondeur de 40 plis lors de l'extension de certaines lignes de recherches
 - ▶ Fonction d'évaluation utilise plus de 8000 caractéristiques
 - ▶ base de données de positions finales, etc.

Jeux déterministes : bref état de l'art

- **Dames** anglaises (checkers) : CHINOOK a battu le champion du monde Marion Tinsley (après 40 ans de titre de champion) en 1994. Depuis 2007, CHINOOK joue de manière optimale grâce à une exploration α - β couplée à une base de données de 39000 milliards de positions de fin de parties. Ce jeu est donc entièrement “résolu” par une machine !

Jeux déterministes : bref état de l'art

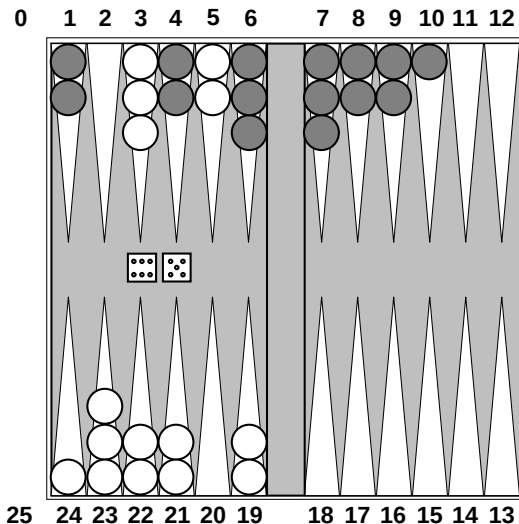
- **Dames** anglaises (checkers) : CHINOOK a battu le champion du monde Marion Tinsley (après 40 ans de titre de champion) en 1994. Depuis 2007, CHINOOK joue de manière optimale grâce à une exploration α - β couplée à une base de données de 39000 milliards de positions de fin de parties. Ce jeu est donc entièrement “résolu” par une machine !
- **Othello** : les champions refusent de jouer contre des ordinateurs, car ceux-ci sont trop bons (l'espace d'états d'Othello est plus réduit que celui du jeu d'Echecs).

Jeux déterministes : bref état de l'art

- **Dames** anglaises (checkers) : CHINOOK a battu le champion du monde Marion Tinsley (après 40 ans de titre de champion) en 1994. Depuis 2007, CHINOOK joue de manière optimale grâce à une exploration α - β couplée à une base de données de 39000 milliards de positions de fin de parties. Ce jeu est donc entièrement “résolu” par une machine !
- **Othello** : les champions refusent de jouer contre des ordinateurs, car ceux-ci sont trop bons (l'espace d'états d'Othello est plus réduit que celui du jeu d'Echecs).
- **Go** : les champions refusaient de jouer contre des ordinateurs, car ceux-ci étaient trop mauvais (à Go, $b > 300$)... jusqu'en 2016 lors d'une partie entre *AlphaGo* (Google) et Lee Sedol (champion du monde).

Jeux de hasard

Jeux non déterministes : backgammon



Jeux non déterministes (en général)

Dans les jeux non déterministes, la chance est introduite par le lancer de dés, le tirage de cartes, etc.

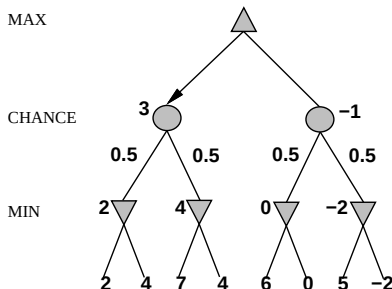
⇒ introduction de noeuds de chance (hasard) et adaptation du Minimax (Algorithme EXPECTIMINIMAX)

Jeux non déterministes (en général)

Dans les jeux non déterministes, la chance est introduite par le lancer de dés, le tirage de cartes, etc.

⇒ introduction de noeuds de chance (hasard) et adaptation du Minimax (Algorithme EXPECTIMINIMAX)

Exemple simplifié (avec lancer d'une pièce) :



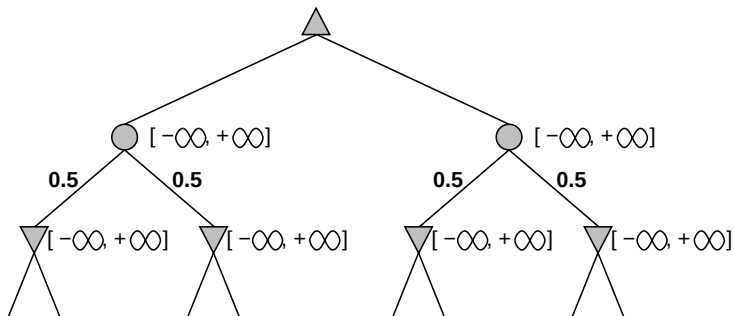
Algorithme pour des jeux non déterministes

EXPECTIMINIMAX : variante de MINIMAX, avec gestion des noeuds de chance où l'on calcule la **moyenne** des valeurs des fils.

```
function EXPECTIMINIMAX(state) returns an action  
...  
if state is a MAX node then  
    return the highest EXPECTIMINIMAX value of ACTIONS(state)  
if state is a MIN node then  
    return the lowest EXPECTIMINIMAX value of ACTIONS(state)  
if state is a chance node then  
    return average of EXPECTIMINIMAX value of ACTIONS(state)  
...
```

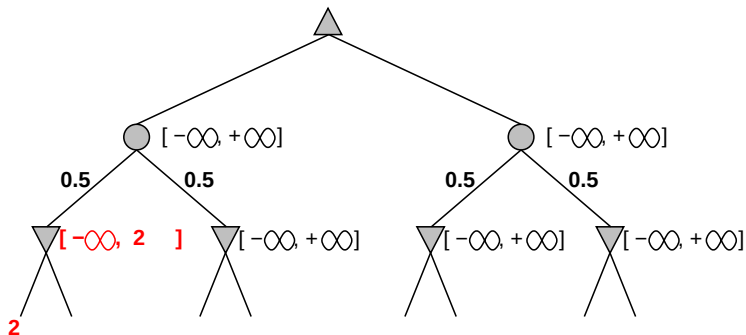
Elagage dans les arbres de jeux non déterministes

Adaptation (illustrée) de l'algorithme α - β :



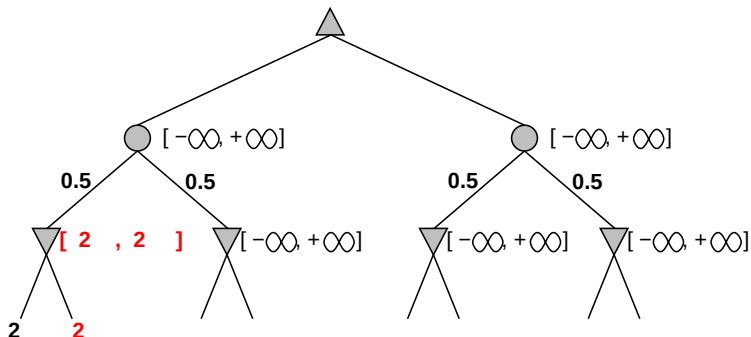
Elagage dans les arbres de jeux non déterministes

Adaptation (illustrée) de l'algorithme α - β :



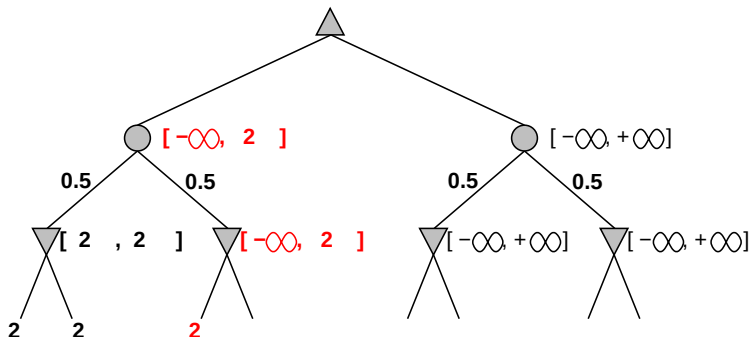
Elagage dans les arbres de jeux non déterministes

Adaptation (illustrée) de l'algorithme α - β :



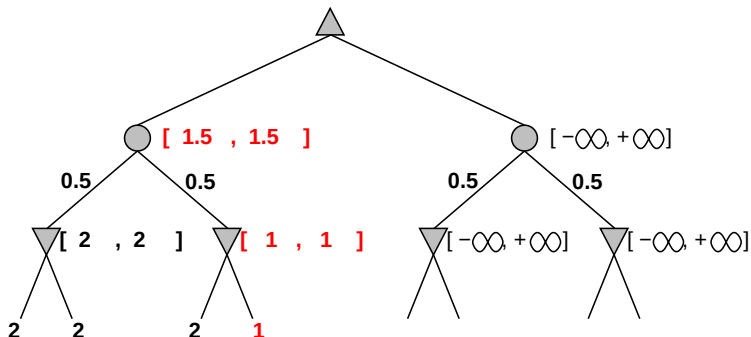
Elagage dans les arbres de jeux non déterministes

Adaptation (illustrée) de l'algorithme α - β :



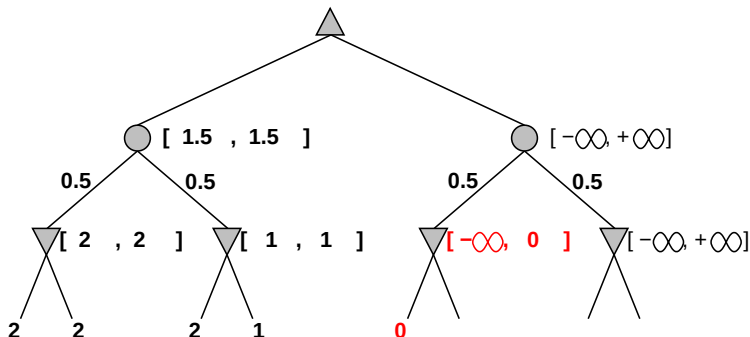
Elagage dans les arbres de jeux non déterministes

Adaptation (illustrée) de l'algorithme α - β :



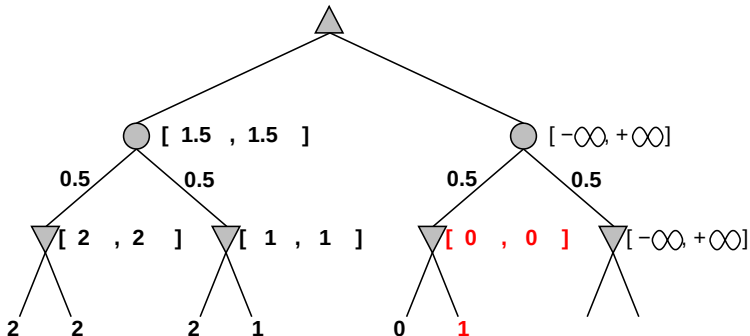
Elagage dans les arbres de jeux non déterministes

Adaptation (illustrée) de l'algorithme α - β :



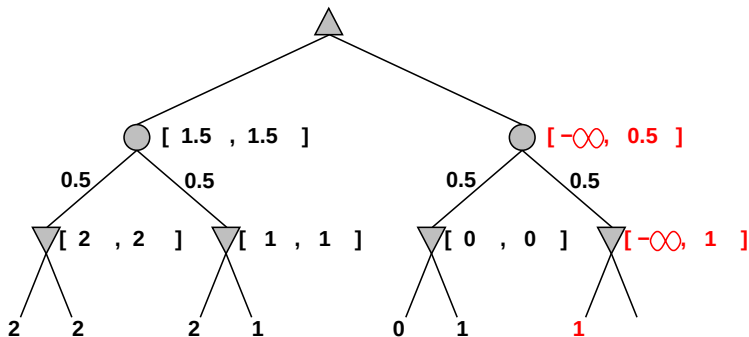
Elagage dans les arbres de jeux non déterministes

Adaptation (illustrée) de l'algorithme α - β :



Elagage dans les arbres de jeux non déterministes

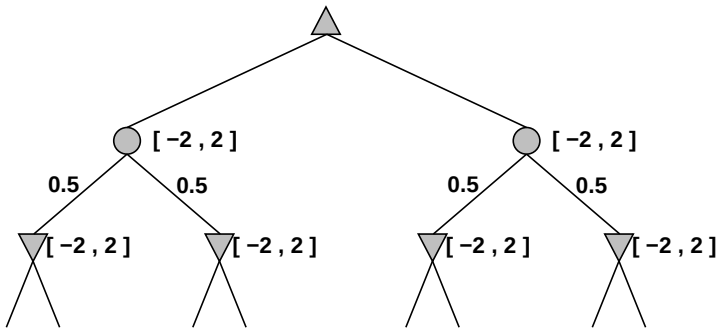
Adaptation (illustrée) de l'algorithme α - β :



Inutile d'aller plus loin.

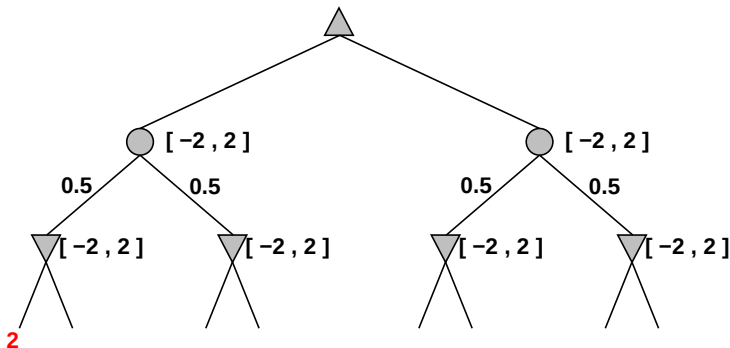
Elagages supplémentaires

Dans la plupart des cas on peut borner les noeuds *a priori*. Par exemple : utilité incluse entre 2 valeurs par les règles du jeu. Des élagages supplémentaires sont alors possibles.



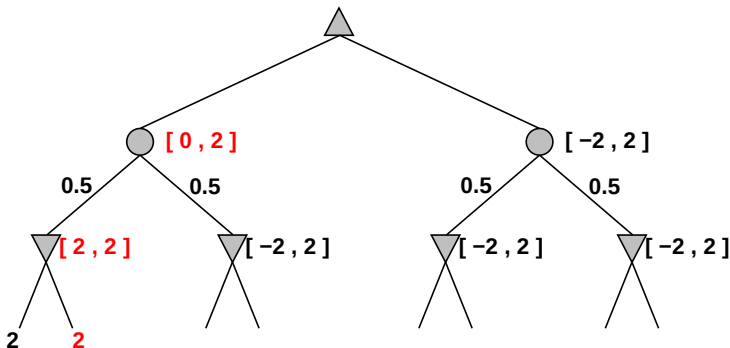
Elagages supplémentaires

Dans la plupart des cas on peut borner les noeuds *a priori*. Par exemple : utilité incluse entre 2 valeurs par les règles du jeu. Des élagages supplémentaires sont alors possibles.



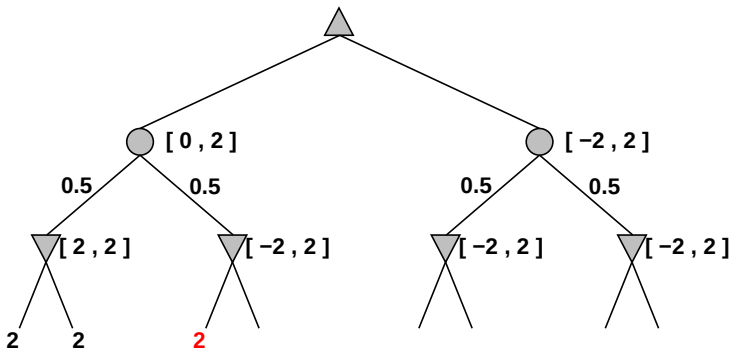
Elagages supplémentaires

Dans la plupart des cas on peut borner les noeuds *a priori*. Par exemple : utilité incluse entre 2 valeurs par les règles du jeu. Des élagages supplémentaires sont alors possibles.



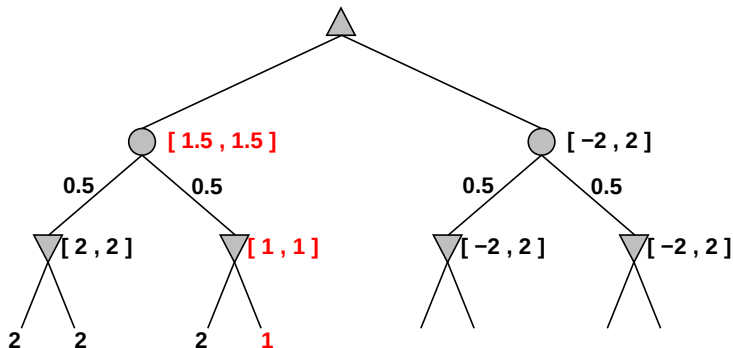
Elagages supplémentaires

Dans la plupart des cas on peut borner les noeuds *a priori*. Par exemple : utilité incluse entre 2 valeurs par les règles du jeu. Des élagages supplémentaires sont alors possibles.



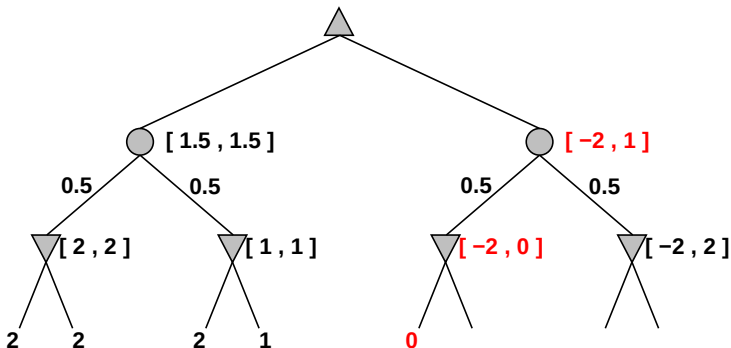
Elagages supplémentaires

Dans la plupart des cas on peut borner les noeuds *a priori*. Par exemple : utilité incluse entre 2 valeurs par les règles du jeu. Des élagages supplémentaires sont alors possibles.



Elagages supplémentaires

Dans la plupart des cas on peut borner les noeuds *a priori*. Par exemple : utilité incluse entre 2 valeurs par les règles du jeu. Des élagages supplémentaires sont alors possibles.



Inutile d'aller plus loin.

Jeux non déterministes en pratique

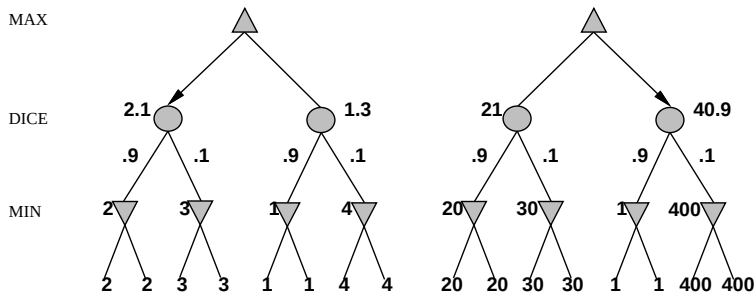
En pratique, le hasard **augmente le facteur de branchement**. Par exemple, avec 2 dés $\Rightarrow$ 21 combinaisons possibles.

Backgammon $\approx$ 20 mouvements autorisés (cela peut être 6000 avec un double “1”).

$$\text{profondeur 4} = 20 \times (21 \times 20)^3 \approx 1.2 \times 10^9$$

Le programme TDGAMMON utilise une exploration de profondeur 2 et très bonne fonction EVAL $\simeq$ niveau champion du monde.

Fonctions d'évaluation : les valeurs exactes ont de l'importance



Le comportement est préservé seulement pour toute transformation **linéaire positive** de EVAL.

Dans le cas de jeux **non déterministes**, EVAL doit être proportionnelle au profit espéré.

Fonctions d'évaluation : Monte Carlo

Pour les jeux de chances, une bonne fonction EVAL n'est donc pas toujours facile à déterminer.

Alternative : réaliser une [simulation de Monte Carlo](#) pour évaluer la qualité des positions en jouant (contre soi-même) des milliers de parties.

Jeux à information imparfaite

Jeux à information imparfaite

L'information est **imparfaite** quand certains éléments sont inconnus.

Par exemple, dans les jeux de cartes : cartes (initiales) des adversaires inconnues.

Jeux à information imparfaite

On peut calculer une probabilité pour chaque possibilité (similaire à un grand **tirage aléatoire** au début du jeu).

Idée : calculer la valeur minimax de chaque action pour chaque tirage et choisir l'action qui a la plus haute valeur espérée pour tous les tirages .

Cas spécial : si une action est optimale pour tous les tirages, elle est optimale .

Jeux à information imparfaite

On peut calculer une probabilité pour chaque possibilité (similaire à un grand **tirage aléatoire** au début du jeu^{*}).

Idée : calculer la valeur minimax de chaque action pour chaque tirage et choisir l'action qui a la plus haute valeur espérée pour tous les tirages^{*}.

Cas spécial : si une action est optimale pour tous les tirages, elle est optimale^{*}.

^{*} **Intuition** : la valeur d'une action est la moyenne des valeurs possibles. Est-ce toujours correct ?

Intuition correcte ?

Jour 1

- Route A mène à un petit tas de pièces d'or
- Route B mène à un croisement :
 - ▶ prenez à gauche et vous trouverez un monticule de bijoux
 - ▶ prenez à droite et un bus vous écrasera

⇒ *B* est le bon choix

Intuition correcte ?

Jour 1

- Route A mène à un petit tas de pièces d'or
- Route B mène à un croisement :
 - ▶ prenez à gauche et vous trouverez un monticule de bijoux
 - ▶ prenez à droite et un bus vous écrasera

⇒ B est le bon choix

Jour 2

- Route A mène à un petit tas de pièces d'or
- Route B mène à un croisement :
 - ▶ prenez à gauche et un bus vous écrasera
 - ▶ prenez à droite et vous trouverez un monticule de bijoux

⇒ B est le bon choix

Intuition correcte ?

Jour 3

- Route A mène à un petit tas de pièces d'or
- Route B mène à un croisement :
 - ▶ devinez la bonne direction et vous trouverez un monticule de bijoux
 - ▶ prenez l'autre direction et un bus vous écrasera

Intuition correcte ?

Jour 3

- Route A mène à un petit tas de pièces d'or
- Route B mène à un croisement :
 - ▶ devinez la bonne direction et vous trouverez un monticule de bijoux
 - ▶ prenez l'autre direction et un bus vous écrasera

⇒ Situation identique au Jour 1 ou au Jour 2

Selon l'intuition "la valeur d'une action est la moyenne des valeurs possibles", il faudrait choisir la route B. Et vous, que choisissez-vous ?

Rationalité dans un monde partiellement observable

Avec une information imparfaite, la valeur d'une action dépend d'**états de croyance** (*belief state*) dans lequel l'agent se trouve.

Un état de croyance $\simeq$ ensemble de tous les états possibles dans lequel un agent pourrait être.

On peut générer et explorer un arbre avec des états de croyance.

Mène à des comportements rationnels tel que :

- Agir pour obtenir de l'information ;
- Faire un signal à un partenaire ;
- Agir aléatoirement pour éviter de divulguer de l'information ;
- etc.

Jeux à information imparfaite en pratique

- **Bridge** : GIB a remporté en 2000 le “computer bridge championship” en utilisant une simulation de Monte Carlo.
Avantage : joueurs humains
- **Poker** : Par ex. : LOKI (Université d'Alberta) également basé sur des simulations.
Avantage : joueurs humains (de loin)

Résumé du chapitre

Résumé du chapitre

Divers jeux ont été présentés pour comprendre comment jouer de manière optimale (en théorie) et comment jouer correctement (en pratique).

- l'algorithme **Minimax** sélectionne les actions optimales, quand l'information est parfaite et le jeu est déterministe
- l'algorithme **α - β** permet d'élaguer certaines branches de l'arbre exploré par Minimax, sans perdre la garantie d'optimalité
- quand la perfection (optimalité) est impossible $\Rightarrow$ on doit **approximer** (fonctions EVAL et CUTOFF-TEST)
- une bonne fonction d'évaluation nécessite une bonne connaissance
- beaucoup de programmes **pré-calculent** les meilleurs coups de début et de fin de parties ("lookup" plutôt qu'exploration)

Résumé du chapitre

- le **hasard** est géré par une extension du Minimax qui utilise des **noeuds de chance**
- jouer de manière optimale avec une **information imparfaite** requiert de raisonner à propos d'**états de croyance** courants et futurs pour chaque joueur. Une simple approximation consiste à prendre la valeur moyenne parmi toutes les valeurs de toutes les actions possibles.

Russel : “Games are to AI as grand prix racing is to automobile design”